The Talent Hack

Hackathon Problem Statements

Deutsche Telekom Digital Labs

Problem Statement 1

Autonomous AI Coding Loop

Build the machinery that makes long-running, model-written coding work trustworthy — where models generate and deterministic checks accept. Participants build one shared core loop, then complete it through one of two tracks.

The Challenge

Long coding tasks rarely succeed with a single prompt. Writing code was never the bottleneck — the bottleneck is everything wrapped around the code: understanding the objective, honouring conventions, proving correctness, recovering from failure, and knowing when to stop and ask a human. Copilots help a developer type faster. They do not deliver.

Your challenge is to build an autonomous AI coding loop that can:

1. Understand the desired outcome.
2. Define what success means.
3. Create a plan.
4. Make changes.
5. Validate the result.
6. Use feedback to try again.
7. Stop when the task succeeds or reaches a safe limit.

We are not asking whether you can prompt a model into producing code. We know you can. We are asking whether you can build the machinery that makes model-written code trustworthy enough to merge.

Shared Core Requirements (Both Tracks)

Whichever track you choose, your system must demonstrably do the following:

- **Plan like a senior engineer.** Turn a prose requirement or objective into a concrete, grounded plan: which modules, which entities, which existing patterns to follow, what order of work, what could go wrong. A plan that names real files beats a plan that names ideas.
- **Separate generation from acceptance.** This is the heart of the challenge. Language models are eloquent about code that does not work. Generation is where models are free; acceptance is where they are forbidden. Acceptance belongs to executable checks — the compiler, the linter, tests, static analysis, migration apply-and-rollback. A change advances only when the checks say so. An LLM's opinion, however confident, must never turn a red check green. Given the same state, the acceptance verdict must be identical every time.
- **Feed failure back into the loop.** When validation fails, the system must produce actionable feedback, pass it into the next attempt, and increase the iteration count.
- **Stay bounded — recover or retreat cleanly.** Retries operate within a budget you define. When the budget runs out, the system must roll the work back, leave the workspace in a green state, and report the work honestly as undelivered. Unlimited autonomous execution is not acceptable. A partial delivery with an honest report is professional; a broken branch is disqualifying.
- **Know when to ask instead of answer.** When a requirement is ambiguous or self-contradictory, the system that notices, halts, and emits a structured clarification request earns credit. The system that confidently implements a guess earns nothing.
- **Keep receipts.** Every run must leave an inspectable trail: which component did what, which files it touched, commands and tools used, what each verification step concluded, retry reasons, and what it all cost — model interactions, iterations, and wall-clock time. If we ask "why did the system merge this?" the answer must be a log, not a shrug.

- **Integrate a real agent runner.** Use at least one coding-agent harness or agent SDK (e.g., OpenCode, Strands Agents, DeepAgents, Aider, Claude Code, Codex, or equivalent) with file-system capabilities: reading, creating, editing, and deleting files; searching repository contents; running shell commands; executing tests, builds, and validation scripts; inspecting diffs; and capturing the agent transcript and execution status. Model, provider, and execution backend should be configurable rather than tightly coupled to the orchestration logic.

Validation may combine deterministic checks (builds, unit and integration tests, linting, type checking, static analysis, coverage thresholds, file checks, API or schema validation, performance checks), LLM-as-a-judge for qualitative areas (architectural alignment, readability, documentation quality, adherence to conventions — with a clear rubric and structured evidence), and human review for high-risk changes, security-sensitive code, public API changes, database changes, and final approval.

Choose one track below. The core loop is common to both; the tracks differ in what surrounds it.

Track A — Feature Factory: Autonomous Delivery on a Brownfield Codebase

Build an autonomous delivery system that ships real features on a codebase it has never seen — and prove you broke nothing.

The Scenario

It is Monday morning at a 400-person software company. The engineering backlog holds ninety product requirements. The team can clear perhaps twelve this quarter. The CTO asks a question that every serious engineering organization is now asking:

“If an AI system can write code, why am I still measuring delivery in quarters?”

The honest answer is that writing code was never the bottleneck. The bottleneck is everything wrapped around the code: understanding a five-year-old codebase, honouring its conventions, not breaking the other 200 features, proving correctness, and knowing when to stop and ask a human. Copilots help a developer type faster. They do not deliver features.

Your challenge is to close that gap. Over 24 hours, you will build a **Feature Factory**: a software system that accepts a product requirement as input and delivers a finished, verified pull request on a large existing codebase as output — with no human hands on the keyboard in between.

We are not asking whether you can prompt a model into producing code. We know you can. We are asking whether you can build the *machinery* that makes model-written code trustworthy enough to merge.

What you do get:

Greenfield demos prove very little, so your factory operates on a brownfield: **Twenty** (github.com/twentyhq/twenty), the open-source CRM, at a commit hash we announce at kickoff. It is a genuinely industrial codebase — NestJS and GraphQL on the backend, React on the front, PostgreSQL underneath, a metadata engine for custom objects, multi-workspace tenancy, and an established design system. Roughly the complexity of the client systems our engineers live in every day.

There is no starter kit. Nobody hands you a fork, a database, or a server. You clone the repository, get it building, seed it with data, and deploy it — yourselves, on the clock. A team that cannot stand up an unfamiliar brownfield codebase unaided has answered one of our hiring questions early.

What you do get:

- **Cursor access, identical for every team.** Use it interactively while you build, and drive it programmatically (headless / CLI agents) inside your factory. Because the AI tooling is uniform, it cannot be your edge. Your edge is the factory.
- **A seed specification.** Your database must contain at least two tenant workspaces (remember this detail — it will matter), 10,000 people, 2,000 companies, and 5,000 opportunities, with the field coverage the spec lists. How you generate that data is your business.
- **A demo-day contract.** Before judging you register a URL for your running deployment and an admin credential. Our hidden acceptance suite logs in, creates its own fixtures through the app's own APIs, tests, and cleans up after itself. An environment we cannot reach is an environment we cannot score.

Everything else — laptops, cloud accounts, deployment targets — you bring.

What the Factory Must Do

We deliberately do not prescribe an architecture. Single orchestrator or a society of agents, one model or five, an agent framework or hand-rolled orchestration — your call, and defending that call is part of the judging. Whatever you build, it must demonstrably do six things:

1. Read the requirement like a senior engineer. Turn Feature requireemnts into a concrete plan grounded in the actual repository: which modules, which entities, which existing patterns to follow, what order of work, what could go wrong. A plan that names real files beats a plan that names ideas.

2. Write code that belongs. Migrations, backend logic, resolvers, UI — produced as commits on a feature branch, indistinguishable in style from code already in the repo. Twenty has conventions for permissions, module layout, and components; code that fights the house style is a defect even when it runs.

3. Never let a model declare victory. Here is the heart of the challenge. Language models are persuasive about code that does not work. So your factory must separate two concerns: generation, where models are free, and acceptance, where they are forbidden. Acceptance belongs to executable checks — the compiler, the linter, the new tests, the repo's full existing suite, migration apply-and-rollback, and tenancy probes. A change advances only when the checks say so. An LLM's opinion, however confident, must never turn a red check green. Given the same branch state, your acceptance verdict must be identical every time.

4. Recover, or retreat cleanly. When verification fails — and it will — the factory diagnoses and retries within a budget you define. When the budget runs out, it must roll the work back and report the story as undelivered, leaving the branch green. A partial delivery with an honest report is professional; a broken branch is disqualifying.

5. Know when to ask instead of answer. One story in the unseen requirement will be quietly self-contradictory. The factory that notices, halts that story, and emits a structured clarification request earns credit. The factory that confidently implements a guess earns whatever the hidden tests give a wrong guess: nothing.

6. Keep receipts. Every run must leave a trail: which component did what, which files it touched, what each verification step concluded, and what it all cost — model interactions, iterations, and wall-clock time, per story and per stage. If we ask "why did the factory merge this?" the answer must be a log, not a shrug.

The Practice Requirement: Lead Scoring & Routing

- Your factory — not your team — implements this on Twenty. Commit history and run logs must make that provable; hand-written feature code disqualifies the story.
- **FEAT-101 · Scoring rules.** Workspace admins define scoring rules through a settings page and the GraphQL API. A rule targets a field on Person, Company, or Opportunity; applies a condition (equals, contains, greater-than, is-empty); and carries a point weight, possibly negative. Non-admins can read, never write. Rules referencing nonexistent fields are rejected with a useful error.
- **FEAT-102 · Live scores.** Each Person shows a Lead Score (sum of matching rule weights) and a Grade — A ≥ 80, B ≥ 50, C ≥ 20, D otherwise — on the record page and as sortable list columns. Scores refresh automatically when the person, their company, or any rule changes. Recomputing all 10,000 people your seed contains must finish without timing out.
- **FEAT-103 · Round-robin routing.** The moment a Person reaches Grade A, they are assigned to a rep drawn round-robin from an admin-defined pool, where each member carries a weekly capacity. Members at capacity are skipped; when everyone is full, leads wait in a visible queue that drains as capacity frees. Every assignment is stamped on the record. Two leads hitting Grade A in the same instant must not be assigned to the same last slot — yes, we test that.
- **FEAT-104 · Tamper-proof audit.** Score changes, grade changes, and assignments append to an audit log carrying old value, new value, the rules that fired, timestamp, and actor. No API path may modify or delete an entry — our suite will try. The log filters by record and by date range, and writing it must not sink FEAT-102's performance.
- **FEAT-105 · The bug ledger.** Not everything in the requirement is new construction. Have a look at existing issues (https://github.com/twentyhq/twenty/issues) and check how your factory is solving them.
- Ship at least three of the four feature stories; the bug ledger is mandatory for everyone.

We Will Try to Break It

- **Live judges at the keyboard.** In the demo, judges type the inputs a new Feature requirements and would see how it runs.

Track B — Loop Engineering Platform: A Control Plane for AI Coding Work

Build a platform that allows engineers to create, run, and inspect AI coding loops — a credible control plane for long-running AI coding work, not another chat interface.

The Platform

Build a platform that allows an engineer to define, configure, execute, and inspect an AI-assisted workflow for completing a non-trivial coding task. The platform must provide a default loop with four roles:

- **Success Criteria Agent —** Converts the engineering objective into measurable completion criteria.
- **Planning Agent —** Creates or revises the implementation plan.
- **Execution Agent —** Makes changes in the codebase.
- **Validation Agent —** Checks whether the task is complete and provides evidence when it is not.

Default loop flow:

Engineering objective → Success Criteria Agent → engineer confirms the contract (or edits criteria) → Planning Agent → Execution Agent → Validation Agent. If validation passes: through a human gate (if required) to Task Successful. If validation fails with evidence: back to the Planning Agent while attempts remain, otherwise Stopped Safely. A rejected human gate also ends in Stopped Safely.

This should be a configurable template, not a fixed hardcoded sequence. Users should be able to adjust: agent instructions; models or coding-agent providers; tools available to each agent; success criteria; validation checks; maximum iterations; failure paths; human approval points; and completion conditions.

The prototype must run one real coding task against a repository — one that requires planning, implementation, validation, and iteration (e.g., adding a feature across multiple files, fixing a complex bug, refactoring a large component, migrating a module, replacing a deprecated dependency, adding tests to an untested module, converting synchronous code to asynchronous, improving performance against a measurable target, or resolving security or static-analysis findings). Keep the task challenging but demonstrable within the event.

The primary user is an engineer or technical lead who has access to a source repository, knows the desired engineering outcome, may not know every implementation step, wants AI agents to work through the task iteratively, needs evidence that the task is complete, and must be able to inspect, stop, edit, and approve the process.

Success Criteria Definition

The platform should support three ways of defining success: user-defined (the engineer writes the criteria), agent-generated (the agent proposes them), and hybrid (the engineer provides the objective and constraints; the agent proposes measurable criteria for approval). Hybrid should be the default. Before execution, the engineer should be able to review, edit, prioritize, and confirm the criteria, including any human approvals.

Possible criteria include: the project builds successfully; existing tests continue to pass; new tests are added; a required file or feature exists; protected files are not modified; coverage reaches a threshold; performance improves against a baseline; public APIs remain compatible; the implementation follows the requested architecture.

Build the Loop Visually

The core experience should be a visual node canvas inspired by tools such as Rivet — a graph-editor layout rather than a wizard or dashboard made from separate forms. The user should be able to add nodes from a small node library, connect nodes to define execution order, select a node and edit its settings, configure success and failure paths, run the complete graph, and see the status and output of each node.

Minimum node library:

- **Input —** provides the coding objective and constraints.
- **Agent —** performs criteria creation, planning, or implementation.
- **Command —** runs a build, test, or repository command.
- **Validator —** decides whether defined criteria are satisfied.
- **Decision —** chooses the next path based on a result.
- **Human Gate —** pauses for approval or feedback.
- **Success / Stop —** completes or safely terminates the run.

The main screen should contain five areas: a top bar (workflow name; save and export; run, pause, and stop controls; run status; attempt counter), the node library panel, the graph canvas (each node showing name, type, status, configuration summary, connection points, and latest result during a run), a node inspector (name, instructions, model, tools, command, validation criteria, retry limit, timeout, success and failure paths — only settings relevant to the selected

node), and a collapsible run console (agent messages, commands executed, files changed, validation evidence, errors, retry reasons, human feedback; clicking a completed or failed node inspects what happened during its execution).

The canvas does not need advanced graph-editing features such as nested graphs, automatic layout, collaboration, or hundreds of node types.

Session Inspection & Configuration as Code

The user must be able to inspect what each agent did — at minimum: agent input, agent output, commands and tools used, files created or changed, validation results, retry reason, and current status. Live token streaming is optional.

The workflow created in the UI should be exportable as YAML or JSON, describing the agents, their instructions and tools, the task contract, validation checks, retry limits, workflow transitions, and approval requirements. Importing and rerunning the same configuration is desirable.

Required Demonstration (Track B)

1. A coding objective being entered.
2. Success criteria being generated or edited.
3. The four-agent loop being configured.
4. The workflow being saved or exported.
5. An execution attempt running against a repository.
6. A real validation result, with evidence visible.
7. A failed attempt feeding back into another iteration.
8. A later success, or a safe stop after attempts are exhausted.
9. Agent sessions and file changes being inspectable.

A deliberately failed first attempt may demonstrate the loop. Recommended demo duration: five to seven minutes.

Bonus Capabilities (Track B)

- Token, model-call, or cost tracking
- Live agent or command streaming
- Git worktrees, containers, or isolated attempts
- Pause, resume, or checkpoint recovery
- Reusable versioned templates
- External coding-harness or multi-provider integration

Desired Outcome (Both Tracks)

A small but credible autonomous AI coding system: engineers define outcomes instead of micromanaging prompts; agents work through configurable roles; validation drives the next iteration; every run leaves inspectable evidence; autonomy remains bounded; and the same loop can be rerun. The winning solution makes long-running AI coding work feel controlled, measurable, and repeatable.

Problem Statement 2

Voice-Controlled Computer Use Agent

An AI agent that understands spoken instructions and safely operates a local desktop environment to complete multi-step tasks.

Context

The agent should behave like a reliable computer assistant rather than a simple voice-command interface. It must understand the user's intent, plan and execute actions across applications, verify that the requested outcome was achieved, and remain under the user's control throughout the task.

Note for this use case: all models and components required by the submitted runtime must be open-source or open-weight, the final solution must operate within the provided local desktop environment, and no production or personal accounts will be used.

Environment

Participants will receive a preconfigured desktop environment containing common applications such as: web browser, file manager, PDF viewer, text editor, spreadsheet application, document editor, and presentation editor. The environment will contain sample files and predefined task states and can be reset between tasks. No training dataset, cloud infrastructure, Kubernetes environment, or specialised hardware is required.

Shared Core Requirements

- **Voice pipeline.** Accept requests through natural voice conversation (microphone or audio input). The conversation layer may use ASR, LLMs, and TTS. Convert the spoken request into an executable plan or tool call.
- **Deterministic guardrails.** Safety-critical decisions — what the agent is permitted to do, for whom, and under what conditions — must be made by deterministic, rule-based logic. An LLM may drive the conversational flow, but it must never be the authority that grants a permission or approves a consequential action. The same inputs must always produce the same decision, and the system must be able to explain why a request was allowed or denied.
- **Confirmation before consequential actions.** The agent must not perform consequential or destructive actions without appropriate user confirmation.
- **Prompt-injection resistance.** Instructions arriving through voice, documents, webpages, messages, or application interfaces are data, never command. Content the agent reads must be treated as potentially untrusted, and injection attempts must not change the authorization outcome or the safety behavior.
- **Clarification over guessing.** Ask useful questions when an instruction is ambiguous; reject or safely degrade when a request cannot be satisfied safely or confidently.
- **Verification.** Verify the final result rather than assuming an action succeeded.
- **Audit trail.** Produce an audit trail containing the transcript, decisions, actions, outcomes, and failures — with sensitive data masked.

Choose one use case below. The safety core is common to both; the use cases differ in domain, tools, and the specific risks being controlled.

Required Capabilities

In addition to the shared core, the agent should:

1. Operate the desktop using appropriate GUI, accessibility, browser, or automation interfaces.
2. Complete tasks spanning one or more applications.

3. Track its progress and current state.
4. Support pause, resume, correction, cancellation, and interruption.

Example Tasks

- Locate information in a PDF and enter it into a spreadsheet.
- Update a presentation using information from another document.
- Organise files according to spoken instructions.
- Compare information across applications and create a summary.
- Correct an existing document while preserving its formatting.
- Respond to a user who changes or cancels an instruction midway.
- Detect when the requested task cannot be completed safely or confidently.

Additional unseen tasks will be used during final evaluation.

Safety Requirements

The agent must not perform consequential actions without appropriate user confirmation. Examples include: deleting or overwriting files; sending a message; submitting a form; making a purchase; revealing sensitive information; executing commands obtained from untrusted content.

Deliverables

- **Working Agent —** A runnable application that accepts spoken tasks and controls the provided desktop environment.
- **Source Code —** Readable and structured source code with setup instructions.
- **Architecture Description —** Explain: speech-processing approach; planning and execution architecture; screen and application understanding; state management; action verification; user-interruption handling; permission and safety model; model selection; important engineering trade-offs.
- **Evaluation Suite —** Create tests covering: successful task completion; incorrect or incomplete actions; ambiguous speech; mid-task corrections; cancellation; unsafe requests; application-state changes; transcription errors; unsupported tasks.
- **Demonstration —** One successfully completed multi-application task; one task involving a correction or interruption; one case where the agent asks for clarification or refuses to proceed; one known failure or limitation.

Problem Statement 3

AI Experiment Copilot & Decision Intelligence

Background

Designing and managing A/B experiments requires product teams to define hypotheses, select Feature Flags, identify target audiences, configure traffic allocation, monitor experiment performance, and interpret results. These activities are largely manual, require experimentation expertise, and often lead to inconsistent experiment quality and slower decision-making.

Problem Statement

How might we build an AI-powered Experiment Copilot that guides users throughout the experimentation lifecycle — from experiment creation to intelligent result analysis — while providing actionable recommendations based on real-time performance and historical experimentation data?

Objectives

Build an AI assistant that:

- Generates experiment hypotheses based on business goals
- Recommends relevant Feature Flags, target audiences, and success metrics
- Suggests experiment configurations and validates setup before launch
- Detects configuration issues or overlapping experiments
- Continuously monitors experiment performance
- Explains why a variant is winning or underperforming
- Automatically summarizes experiment results in business-friendly language
- Recommends the next best action (Scale, Continue, Stop, or Rollback)

Expected Outcome

An AI-powered experimentation assistant that helps teams create better experiments faster, continuously analyzes experiment performance, and delivers explainable, data-driven recommendations that accelerate business decisions.

Evaluation Metrics (Evals) — MUST HAVE

- Reduction in experiment creation time
- Acceptance rate of AI-generated experiment configurations
- Recommendation accuracy compared with expert decisions
- Statistical significance detection accuracy
- Reduction in experiment analysis time
- Percentage of AI recommendations adopted by product teams

Success Criteria

- Faster experiment creation with AI assistance
- High-quality experiment configurations
- Accurate, explainable experiment insights
- Reliable recommendations for Scale, Continue, Stop, or Rollback
- Reduced manual effort across the experimentation lifecycle

Problem Statement 4

Build a Configurable Decision Automation Platform

Background

Organizations frequently need systems that evaluate incoming requests against configurable business rules and produce decisions with clear explanations. These systems are used across domains such as finance, insurance, healthcare, e-commerce, HR, logistics, and customer support.

Your task is to build a backend application that can process requests, evaluate configurable rules, generate decisions, and expose APIs for external integration.

The emphasis of this challenge is how effectively you leverage AI tools throughout the software development lifecycle, rather than simply producing working code.

Objective

Design and implement a service that:

- Accepts structured requests
- Evaluates configurable business rules
- Produces one or more decisions
- Explains how each decision was reached
- Exposes REST APIs
- Is designed for extensibility, maintainability, and production readiness

Functional Requirements

1. Rule Management

Support configurable rules without requiring code changes. Example rule types:

- Numeric comparisons
- Boolean conditions
- String matching
- Date comparisons
- Multiple conditional expressions
- Rule priorities

Rules may be stored in JSON, YAML, or a database.

2. Decision Engine

The engine should: evaluate incoming requests; execute applicable rules; produce one or more outcomes; return confidence or priority scores where applicable; and explain why each decision was made.

3. Explainability

Every response should include: rules evaluated, rules matched, rules rejected, the final decision, and a human-readable explanation.

4. Extensibility

The platform should allow new rule types or decision logic to be added with minimal code changes.

Non-Functional Requirements

The solution should demonstrate:

- Clean architecture
- Modular design
- Proper separation of concerns
- Logging
- Error handling
- Configuration management
- Unit tests
- API documentation
- Containerization (Docker preferred)

AI Usage Requirement

Candidates are expected to use one or more AI development tools (e.g., ChatGPT, GitHub Copilot, Cursor, Claude, Gemini). Along with the solution, submit an AI Engineering Log describing:

- AI tools used
- Key prompts provided
- AI-generated code that was accepted
- AI-generated code that was rejected or modified
- How AI outputs were validated
- Bugs or issues introduced by AI and how they were resolved

Deliverables

- Source code repository
- README with setup instructions
- Architecture overview
- API documentation
- Unit tests
- AI Engineering Log

Mid-Challenge Change Request (Provided During the Interview)

After approximately half the allotted time, interviewers introduce a new requirement. Examples include:

- Support a new rule type
- Add rule versioning
- Introduce asynchronous processing
- Enable audit history
- Add bulk evaluation
- Integrate an external API
- Add authentication or authorization

The solution should accommodate these changes with minimal disruption to the existing design.

Problem Statement 5

Build an Omnichannel Consumer AI Engine for Digital Commerce

Background

Digital commerce platforms today offer thousands of products, plans, devices, accessories and services. Customers often struggle to discover the most relevant products, resulting in lower engagement, reduced conversions and shopping cart abandonment.

Design and build an AI-powered Omnichannel Consumer Intelligence Engine that personalizes the customer journey and seamlessly integrates across OneShop (Web) and OneApp (Mobile) to improve business outcomes.

The solution should leverage Generative AI, Agentic AI and modern recommendation techniques to deliver intelligent, context-aware experiences throughout the customer journey.

Objective

Build an AI engine that helps customers discover the right products, make better purchase decisions and complete their purchases with minimal friction — regardless of whether they are on the web or mobile. The engine should work consistently across channels and maintain a unified customer experience.

Expected Capabilities

Participants may implement one or more of the following AI capabilities:

- **Personalized Discovery —** Recommend relevant products, plans and accessories based on customer intent.
- **Conversational Shopping Assistant —** Help customers discover, compare and select products using natural language.
- **Next Best Action —** Guide customers through the funnel with contextual recommendations to improve conversion.
- **Smart Cart & Checkout —** Reduce cart abandonment through intelligent nudges, bundle suggestions and checkout assistance.
- **Omnichannel Experience —** Deliver consistent recommendations and customer context across OneShop (Web) and OneApp (Mobile).

Bonus Consideration

Teams/individuals will receive bonus consideration for:

- Multi-agent architecture for personalization and recommendations
- Real-time recommendation engine
- Explainable AI ("Why this recommendation?")
- Continuous learning from customer interactions
- A/B testing framework for AI recommendations
- Voice-enabled shopping assistant